Tic Tac Toe Application

Technical overview (end-to-end) · React 18, Redux, Nx, MUI, json-server · Version 1.0

1. Why this application exists

This is a **frontend-only** Tic Tac Toe product built to a formal technical exercise. It shows how to:

- Separate **UI, state, domain rules, and API** in a maintainable structure.
- Handle modes, undo, scoreboard, and computer AI with **predictable global state (Redux)**.
- Use a **local REST mock** (json-server) without a production .NET/Node backend.
- Organize code in an **Nx monorepo** for review and future growth.

Why no custom backend? Rules, validation, win/draw, undo, and AI stay in the client so reviewers can install and test quickly. The API only **stores JSON**; it does not enforce game rules.

2. What the application provides

Feature	Behaviour
Two-player mode	Users play X and O; X starts; alternating turns.
Computer mode	Human is X; O moves automatically after each valid human move.
Board	3×3; occupied cells locked; no moves after win or draw.
Move history	Move #, player, row/column (1-based).
Undo	Two-player: one move. Computer: human+AI pair when applicable. Disabled after game ends.
Scoreboard	X wins, O wins, draws — once per completed game.
Reset game / scoreboard	Game reset keeps scoreboard; scoreboard reset does not clear the board.
API sync	Persist game and scoreboard via json-server.

3. Technology stack — what and why

Technology	Use	Why
React 18	UI	Functional components, hooks, standard for interactive web apps.
Nx	Workspace	Monorepo, serve/build/test commands, library boundaries.
Redux + thunk	State & async	Single store for game + scoreboard; thunks for API (per spec, no RTK).
MUI	Styling	Buttons, layout, tables, alerts; minimal custom CSS.
json-server	Mock API	REST over db.json for local persistence demo.
Vite	Bundler	Fast dev server under Nx.
Vitest + RTL	Tests	Logic, reducers, smoke UI.

4. Architecture (layers)

```
Browser :4200
  apps/tic-tac-toe          → shell (Redux Provider, MUI theme, init)
  libs/game/feature-game    → GamePage (dispatch, selectors)
  libs/game/ui-board        → GameBoard, BoardCell (presentational)
  libs/scoreboard/feature   → ScoreboardPanel
  libs/game/data-access     → store, reducers, thunks, fetch API
  libs/game/util-game-logic → pure rules, validation, AI, win/draw
        ↓ HTTP fetch
json-server :3000  →  db.json (games[], scoreboard[])
```

Rule: Components do not implement win logic or AI; they dispatch thunks that call pure functions.

5. Project layout (Nx)

```
tic-tac-toe/
  apps/tic-tac-toe/
  libs/game/util-game-logic/
  libs/game/data-access/
  libs/game/ui-board/
  libs/game/feature-game/
  libs/scoreboard/feature-scoreboard/
  db.json
```

6. Redux state

Game slice

Fields include: `gameId`, `board`, `currentPlayer`, `mode`, `status`, `winner`, `winningCells`, `moveHistory`, `isComputerMoving`, `isResultRecorded`, `loading`, `error`.

Scoreboard slice

`xWins`, `oWins`, `draws`, plus loading/error for API calls.

Typical move flow

1. Click cell → `playMove` thunk.
2. `validateMove` → reject invalid moves with message.
3. `applyMoveToBoard` → reducer `APPLY_MOVE`.
4. If finished → `recordGameResultIfNeeded` (scoreboard once).
5. `syncGameState` → PATCH game.
6. Computer mode → `playComputerTurn` if O's turn.

7. Game logic library

Module	Responsibility
board.ts	Empty board; cell index ↔ row/column.
validation.ts	Bounds, turn, mode, occupied cells, game status.
win.ts	Lines, winner, draw.
game-logic.ts	Apply move; rebuild from history (undo).
computer.ts	Win → block → centre → corner → first free cell.

8. Mock API

Method	Endpoint	Purpose
GET	/scoreboard/1	Load scores
PATCH	/scoreboard/1	Update / reset scores
POST	/games	Create game
PATCH	/games/:id	Sync game state

`scoreboard` in db.json must be an **array** so `/scoreboard/1` works. Client validates all moves; server stores JSON only.

9. UI (MUI)

GamePage: mode select, live status, board, Undo / Reset Game / Reset Scoreboard, history table, scoreboard card. Board cells are keyboard-accessible; winning cells use colour and border (not colour alone).

10. Key design decisions

Topic	Choice	Reason
Undo after win/draw	Disabled	No scoreboard rollback.
Mode change	Reset board, keep scoreboard	Per specification.
Scoreboard	isResultRecorded	One update per completed game.

11. How to run (end-to-end)

1. `cd tic-tac-toe && npm install`
2. `npm run api` → port 3000
3. `npm run start` → port 4200 (or `npm run dev` for both)
4. Open **http://localhost:4200** for the game.
5. `npm test` for unit tests.

12. Testing

Covers valid/invalid moves, wins, draws, undo, computer AI priorities, reducer reset/undo, App smoke render. Run: `nx test tic-tac-toe`.

13. Limitations

- Single scoreboard record (id: 1).
- No auth or multi-tenant isolation.
- API required for persistence; game rules always run in the browser.

14. Summary

A **spec-driven, layered** React app: pure logic in **util-game-logic**, state and HTTP in **data-access**, UI in **feature/ui** libraries, persistence via **json-server**. This supports technical review of both behaviour and engineering structure.